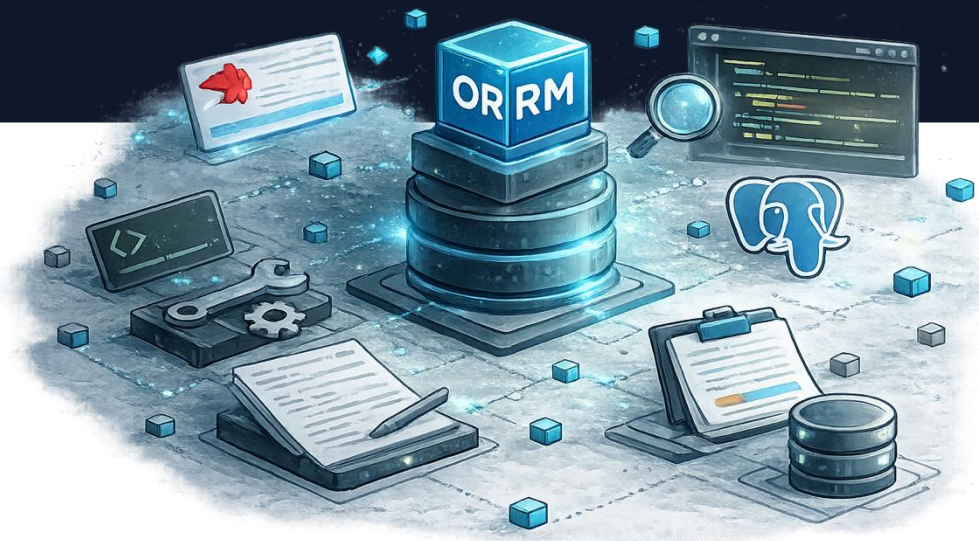




Progettazione di applicazioni web full stack TypeORM e PostgreSQL



Prof. Devis Bianchini
Dip. di Università degli Studi di Brescia



Argomento di questa lezione...

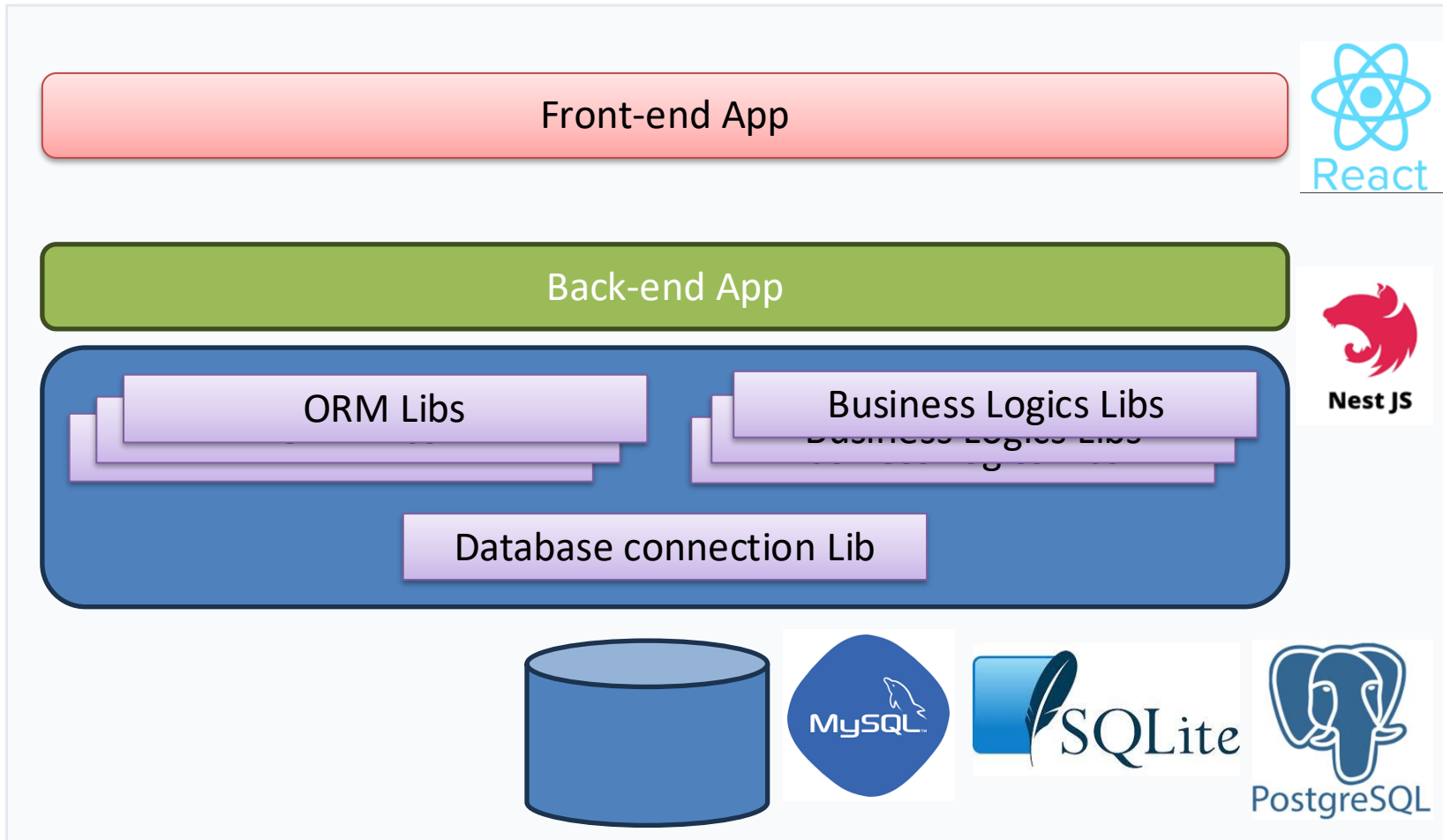
Cosa faremo

- Introduzione a **TypeORM** e aspetti di configurazione
- Concetti di **Entità**, *decorator* TypeORM, **repository**
- Organizzazione dei componenti per l'interazione con i database relazionali in librerie da importare nelle applicazioni

Output

- Una libreria backend in **NestJS** che fa uso degli elementi fondamentali di **TypeORM**
- Estensione di un workspace esistente (**nest-lab2**) tramite integrazione della nuova libreria di backend

Architettura multi-tier (con DB)



Prerequisiti (per PostgreSQL)



- Un DBMS attivo (PostgreSQL in un container Docker)
- Un'interfaccia client per ispezionare il DBMS (per debug - pgAdmin)
- Alcune librerie installate nel workspace Nx

```
npm i @nestjs/typeorm  
npm i typeorm  
npm i pg
```

TypeORM - Definizione



- ***Che cos'è*** → una libreria che permette di lavorare con il database (relazionale) usando classi e oggetti invece di scrivere SQL manualmente (***Object-Relational Mapping***)
- Una classe corrisponde ad una *tabella*, un oggetto corrisponde ad una *riga*, le proprietà della classe corrispondono alle *colonne*
- Fa uso di ***decorators*** del package **typeorm**
- Rende il codice leggibile, riduce il boilerplate SQL, avvicina il modello dei dati al modello ad oggetti

Connessione al DB



- Una libreria dedicata alla configurazione della connessione al database – *Basta un modulo...*

Inizializza **TypeORM** una volta sola, a livello globale/applicazione:

1. Apre la connessione al database **PostgreSQL**
2. Registra **TypeORM** dentro **NestJS**
3. Rende disponibile la connessione agli altri moduli

```
npx nx g @nx/nest:library  
libs/database  
--unitTestRunner=none
```

```
import { Module } from '@nestjs/common';  
import { TypeOrmModule } from '@nestjs/typeorm';  
import 'dotenv/config';  
  
@Module({  
  imports: [  
    TypeOrmModule.forRoot({  
      type: 'postgres',  
      host: process.env.PG_HOST ?? 'localhost',  
      port: Number(process.env.PG_PORT ?? 5432),  
      username: process.env.PG_USERNAME ?? 'postgres',  
      password: process.env.PG_PASSWORD ?? 'postgres',  
      database: process.env.PG_DATABASE ?? 'app',  
      autoLoadEntities: true,  
      synchronize: true,  
    }),  
  ],  
  exports: [TypeOrmModule],  
})  
export class DatabaseModule {}
```

Connessione al DB



- Una libreria dedicata alla configurazione della connessione al database

Variabili d'ambiente utilizzate per la configurazione del DB per l'intero workspace

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import 'dotenv/config';

@Module({
  imports: [
    TypeOrmModule.forRoot({
      type: 'postgres',
      host: process.env.PG_HOST ?? 'localhost',
      port: Number(process.env.PG_PORT ?? 5432),
      username: process.env.PG_USERNAME ?? 'postgres',
      password: process.env.PG_PASSWORD ?? 'postgres',
      database: process.env.PG_DATABASE ?? 'app',
      autoLoadEntities: true,
      synchronize: true,
    }),
  ],
  exports: [TypeOrmModule],
})
export class DatabaseModule {}
```

Connessione al DB



- Una libreria dedicata alla configurazione della connessione al database

- **autoLoadEntities: true** indica che verranno caricate automaticamente tutte le entità registrate nei moduli NestJS (vedi slide successive)
- **synchronize: true** indica che il DB verrà sincronizzato automaticamente con le entità (creazione delle tabelle mancanti, aggiunta delle colonne, aggiornamento della struttura in base alle entità) → non consigliato in produzione

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import 'dotenv/config';

@Module({
  imports: [
    TypeOrmModule.forRoot({
      type: 'postgres',
      host: process.env.PG_HOST ?? 'localhost',
      port: Number(process.env.PG_PORT ?? 5432),
      username: process.env.PG_USERNAME ?? 'postgres',
      password: process.env.PG_PASSWORD ?? 'postgres',
      database: process.env.PG_DATABASE ?? 'app',
      autoLoadEntities: true,
      synchronize: true,
    }),
  ],
  exports: [TypeOrmModule],
})
export class DatabaseModule {}
```


Una libreria per ogni tabella



- Per mantenere il progetto ordinato e scalabile, a ogni tabella del database viene fatta corrispondere una *libreria dedicata* che raccoglie tutto ciò che serve per gestirla

```
npx nx g @nx/nest:library  
libs/server/users --service -controller  
  
npx nx g @nx/nest:class  
libs/server/users/src/lib/users.entity  
[--unitTestRunner=none]  
  
npx nx g @nx/nest:class  
libs/server/users/src/lib/users.repository  
[--unitTestRunner=none]  
  
npx nx g @nx/nest:class  
libs/server/users/src/lib/dto/create-user.dto  
[--unitTestRunner=none]
```

Crea tre file:

- `users.module.ts`
- `users.service.ts` (e corrispondente file di test `.spec.ts`)
- `users.controller.ts` (e corrispondente file di test)

Una libreria per ogni tabella



- Per mantenere il progetto ordinato e scalabile, a ogni tabella del database viene fatta corrispondere una *libreria dedicata* che raccoglie tutto ciò che serve per gestirla

```
npx nx g @nx/nest:library  
libs/server/users --service -controller  
  
npx nx g @nx/nest:class  
libs/server/users/src/lib/users.entity  
[--unitTestRunner=none]  
  
npx nx g @nx/nest:class  
libs/server/users/src/lib/users.repository  
[--unitTestRunner=none]  
  
npx nx g @nx/nest:class  
libs/server/users/src/lib/dto/create-user.dto  
[--unitTestRunner=none]
```

Crea un file:

- **users.entity.ts** per specificare il mapping verso il database (tramite **TypeORM**)

Una libreria per ogni tabella



- Per mantenere il progetto ordinato e scalabile, a ogni tabella del database viene fatta corrispondere una *libreria dedicata* che raccoglie tutto ciò che serve per gestirla

```
npx nx g @nx/nest:library  
libs/server/users --service --controller  
  
npx nx g @nx/nest:class  
libs/server/users/src/lib/users.entity  
[--unitTestRunner=none]  
  
npx nx g @nx/nest:class  
libs/server/users/src/lib/users.repository  
[--unitTestRunner=none]  
  
npx nx g @nx/nest:class  
libs/server/users/src/lib/dto/create-user.dto  
[--unitTestRunner=none]
```

Crea un file:

- **users.repository.ts** per governare l'accesso ai dati nel database (azioni CRUD)
- Possibile usare anche il comando **npx nx g @nx/nest:service** (il repository è a sua volta un **provider**), ma meno controllo sul nome del file (aggiunge il suffisso **.service.ts** al nome)

Una libreria per ogni tabella



- Per mantenere il progetto ordinato e scalabile, a ogni tabella del database viene fatta corrispondere una *libreria dedicata* che raccoglie tutto ciò che serve per gestirla

```
npx nx g @nx/nest:library  
libs/server/users --service -controller  
  
npx nx g @nx/nest:class  
libs/server/users/src/lib/users.entity  
[--unitTestRunner=none]  
  
npx nx g @nx/nest:class  
libs/server/users/src/lib/users.repository  
[--unitTestRunner=none]  
  
npx nx g @nx/nest:class  
libs/server/users/src/lib/dto/create-user.dto  
[--unitTestRunner=none]
```

Nel repository, quando si implementano le azioni **Create** and **Update**, meglio utilizzare un **DTO** (**Data Transfer Object**) per controllare il tipo di dato passato

Una libreria per ogni tabella



- Per mantenere il progetto ordinato e scalabile, a ogni tabella del database viene fatta corrispondere una *libreria dedicata* che raccoglie tutto ciò che serve per gestirla



Una **Entity** con i *decorator*
TypeORM



```
import { Entity, PrimaryGeneratedColumn, Column } from 'typeorm';
import { UserRole } from '../dto/user-role.enum.js';

@Entity('users')
export class UserEntity {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({type: 'varchar', length: 255, nullable: false})
  name: string;

  @Column({type: 'varchar', length: 320, nullable: true})
  email: string | null;

  @Column({
    type: 'enum',
    enum: UserRole,
    default: UserRole.USER
  })
  role: UserRole;
}
```

Altri *decorator* per entità



- Dalla documentazione di **TypeORM**

- **@PrimaryColumn**

- Come **@PrimaryGeneratedColumn** (chiave primaria), ma senza auto-increment

Altri *decorator* per entità



- Dalla documentazione di **TypeORM**

- **@CreateDateColumn**
- **@UpdateDateColumn**
- **@DeleteDateColumn**

- Colonne di tipo **Date** che vengono automaticamente valorizzate quando un'istanza viene creata, aggiornata, cancellata (meccanismo di *soft-delete*)

Altri *decorator* per entità



- Dalla documentazione di **TypeORM**

- `@Column("simple-array")`

- Per colonne che contengono array di valori

Altri *decorator* per entità



- Dalla documentazione di **TypeORM**
- | | |
|---|--|
| <ul style="list-style-type: none">• @Column("simple-json") | <ul style="list-style-type: none">• Per colonne che contengono qualsiasi valore salvabile nel database come una stringa JSON (via JSON.stringify) |
|---|--|

Altri *decorator* per entità



- Dalla documentazione di **TypeORM**

- `type: ColumnType`
- `name: string`
- `length: number`
- `nullable: boolean`
- `default: string`
- `unique: boolean`
- `select: boolean`

- Le opzioni più comuni da utilizzare nel decorator **@Column**
- Interessante l'opzione **select** (**true** per default) che indica che quel campo deve essere compreso a seguito di una **select * from <table>**

Estendiamo al caso di più tabelle



- La scelta di una relazione uno-a-uno tra librerie e tabelle mantiene il codice altamente modulare, ma sposta ai livelli superiori (*service*) la gestione delle *relazioni* tra tabelle e i relativi *vincoli di integrità referenziale*
 - Non vengono utilizzati i decorator `@OneToOne`, `@OneToMany`, `@ManyToMany`
 - Nei casi *uno-a-uno* e *uno-a-molti*, vengono semplicemente utilizzate le colonne di una delle due tabelle collegate (il vincolo di chiave esterna è gestito a livello di service)
 - Nel caso *molti-a-molti*, viene creata un'ulteriore libreria
- Nel prossimo laboratorio vedremo l'uso dei decorator per gestire le relazioni tra tabelle di un database relazionale

Una libreria per ogni tabella



- Per mantenere il progetto ordinato e scalabile, a ogni tabella del database viene fatta corrispondere una **libreria dedicata** che raccoglie tutto ciò che serve per gestirla



Una **Entity** con i *decorator* **TypeORM**



Un **repository** per l'accesso ai dati



- File **users.repository.ts**

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { UserEntity } from './user.entity';
import { Repository } from 'typeorm';
import { CreateUserDto } from './dto/create-user.dto';
import { UpdateUserDto } from './dto/update-user.dto';
import { UserRole } from './dto/user-role.enum';
```

```
@Injectable()
export class UsersRepository {
  constructor(
    @InjectRepository(UserEntity)
    private readonly repository: Repository<UserEntity> {}
```

Una libreria per ogni tabella



- Per mantenere il progetto ordinato e scalabile, a ogni tabella del database viene fatta corrispondere una **libreria dedicata** che raccoglie tutto ciò che serve per gestirla



Una **Entity** con i *decorator* TypeORM



Un **repository** per l'accesso ai dati



Eventuali DTO, interfacce o tipi di dato utili per lavorare con la libreria



```
import { IsEmail, IsEnum, IsNotEmpty, IsString } from "class-validator";
import { UserRole } from './user-role.enum';

export class CreateUserDto {
  @IsString()
  @IsNotEmpty()
  name: string;

  @IsEmail()
  email: string;

  @IsEnum(UserRole, {
    message: 'Valid role required among USER or ADMIN'
  })
  role: UserRole;
};

import { CreateUserDto } from './create-user.dto';
import { PartialType } from "@nestjs/mapped-types";

export class UpdateUserDto extends PartialType(CreateUserDto) {
}
```

Una libreria per ogni tabella



- Per mantenere il progetto ordinato e scalabile, a ogni tabella del database viene fatta corrispondere una **libreria dedicata** che raccoglie tutto ciò che serve per gestirla



Una **Entity** con i *decorator* **TypeORM**



Un **repository** per l'accesso ai dati



Eventuali DTO, interfacce o tipi di dato utili per lavorare con la libreria



La logica specifica collegata alla risorsa rappresentata dalla tabella (service, controller)

- File **users.service.ts**

```
import { Injectable } from '@nestjs/common';
import { CreateUserDto } from '../dto/create-user.dto';
import { UpdateUserDto } from '../dto/update-user.dto';

import { NotFoundException, ConflictException } from '@nestjs/common';

import { UsersRepository } from '../users.repository';
import { UserEntity } from '../user.entity';
import { UserRole } from '../dto/user-role.enum';

@Injectable()
export class ServerUsersService {

  // Injecting the repository
  constructor(private readonly usersRepository: UsersRepository){}
```

- File **users.controller.ts**

Esportare la libreria...



- File `users.module.ts`

```
import { Module } from '@nestjs/common';
import { ServerUsersController } from './users.controller';
import { ServerUsersService } from './users.service';
import { UsersRepository } from './users.repository';
import { UserEntity } from './user.entity';
import { TypeOrmModule } from '@nestjs/typeorm';

@Module({
  imports: [TypeOrmModule.forFeature([UserEntity])],
  controllers: [ServerUsersController],
  providers: [
    ServerUsersService,
    UsersRepository
  ],
  exports: [UsersRepository, ServerUsersService],
})
export class ServerUsersModule {}
```

...per importarla in una qualsiasi app



- Nella app di backend

```
import { Module } from '@nestjs/common';
import { AppController } from './app.controller';
import { AppService } from './app.service';
import { ServerUsersModule } from '@server/users';
import { DatabaseModule } from '@org/database';

@Module({
  imports: [ServerUsersModule, DatabaseModule],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
```




- Link alla documentazione TypeORM:
<https://typeorm.io/docs>
- Integrazione di TypeORM in NestJS:
<https://docs.nestjs.com/techniques/database>



Progettazione di applicazioni web full stack TypeORM e PostgreSQL



```
mirror_mod.use_x = False
mirror_mod.use_y = True
mirror_mod.use_z = False
operation = "MIRROR_Z":
mirror_mod.use_x = False
mirror_mod.use_y = False
mirror_mod.use_z = True

selection at the end -add
_ob.select= 1
ier_ob.select=1
context.scene.objects.active
("Selected" + str(modifier
mirror_ob.select = 0
bpy.context.selected_object
data.objects[one.name].select
print("please select exactly

-- OPERATOR CLASSES -----

types.Operator):
X_mirror to the selected
X_mirror = mirror_x"
```

Prof. Devis Bianchini

Dip. di Università degli Studi di Brescia